Chapitre 4 :
Calculabilité et décidabilité

Sources :
- Carton : Langages formels: Calculabilité et complexité
- Wolper : Introduction à la calculabilité (/!\ Pas très bien traduit)

Table des matières

N.1 Introduction 1

I Problèmes algorithmiques 1

I.1 Fonctions calculables 1

I.2 Problèmes de décision 2

I.3 Problèmes d'optimisations 3

II Modèle de calcul 4

II.1 Modèle de calcul choisi par le programme 4

III Décidabiltié 6

III.1 Définition 6

III.2 Problème de l'arrêt 6

III.3 Réductions 8

N.1 Introduction

Un des buts principaux de l'informatique, c'est résoudre des « problèmes » avec des « algorithmes ».

On se pose plusieurs questions :
1. Est-ce que tout « problème » peut se résoudre avec un « algorithme » ?
2. Pour les « problèmes » qu'on peut résoudre, combien de temps met le « meilleur » algorithme pour s'exécuter ?
3. Que faire si le « meilleur » « algorithme » prend trop de temps ?

Pour la première, la réponse est non, par exemple le 10e problème de Hilbert : soit $ax + by + cz = 0, (a, b, c) \in \mathbb{Z}$ une équation diophantienne, est-ce qu'il y a une solution entière ?

I Problèmes algorithmiques

I.1 Fonctions calculables

Définition

Un **problème algorithmiques** dont les entrées vivent dans un ensemble E et les solutions dans un ensemble S est une relation binaire R sur E×S

🔥 ***Remarque***

Une entrée peut être associée à plusieurs solutions, R n'est donc pas forcément une fonction mais on se concentre souvent sur ce cas.

- En calculabilité, on impose implicitement que E soit infini, sans quoi il suffirait de tabuler toutes les réponses pour résoudre le problème

Définition

Définition : Une fonction $f : E \to S$ est dite **calculable** s'il existe un « algorithme » A tel que pour tout $e \in E$, appliquer A sur e calcule $f(e)$ en temps fini

🔥 ***Remarque***

Fonction mathématique n'est pas équivalent à fonction informatique :

Fonction mathématique	Fonction informatique
Pas de procédure de calcule nécessaire	Description de comment calculer les images nécessaire
Domaine de définition quelconque	Domaines de définitions restreints, par exemple on ne peut pas avoir de fonctions informatiques $\mathbb{R} \to X$ car on ne peut pas représenter chaque réel de manière finie.

I.2 Problèmes de décision

Définition

Un **problème de décision** est une fonction $f : E \to \mathbb{B}$, où $\mathbb{B}$ est l'ensemble des booléens.
Généralement, on exprime un problème de décision en donnant
- Son domaien d'entrées E
- Une question à laquelle on ne peut répondre que par oui ou non.

Exemple de problèmes de décision (1):

$$\text{PREMIER} : \begin{cases} \underline{\text{Entrée :}} \;\; n \in N \\ \underline{\text{Question :}} \; \text{Est-ce que } n \in \mathbb{P} \end{cases}$$

La fonction sous-jacente est $\mathbb{1}_{\mathbb{P}} : \mathbb{N} \to \mathbb{B}$

$$\text{RATIONNEL} : \begin{cases} \underline{\text{Entrée :}} \;\; L \in \text{Alg}(\Sigma^*) \\ \underline{\text{Question :}} \;\; L \in \text{Rat}(\Sigma^*)? \end{cases}$$

$$\text{H10} : \begin{cases} \underline{\text{Entrée :}} \; \text{Une équation dipohantienne (E)} \\ \underline{\text{Question :}} \; \text{Est-ce que (E) admet une solution entière ?} \end{cases}$$

Exemple de problèmes qui ne sont pas des problèmes de décision (2):

$$\text{Gaillac-Grenoble} \begin{cases} \underline{\text{Entrée}} : \text{Le réseau ferroviaire français pondéré} \\ \qquad\qquad \text{par temps de trajet} \\ \underline{\text{Question}} : \text{Est-ce qu'on peut faire le trajet entre} \\ \qquad\qquad \text{Grenoble et Gaillac en moins d'une heure ?} \end{cases}$$

$$\text{PREMIER_R} : \begin{cases} \underline{\text{Entrée :}} \;\; n \in [\![1, A(100,100)]\!] \\ \underline{\text{Question :}} \;\; n \in \mathbb{P} \text{ ?} \end{cases}$$

Définition

Un élément qui fait partie de l'ensemble d'un problème de décision est une **instance** de ce problèmes.
Si la réponse à la question pour une instance est « oui » on dit quelle est **positive**, sinon qu'elle est **négative**.

I.3 Problèmes d'optimisations

Définition

Un **problème d'optimisation** est la donnée d'une relation binaire sur E×S et d'une fonction de coût $c : S \to \mathbb{R}$

Généralement, pour introduire un problème d'optimiation on donne
- Son ensemble d'entrées
- À quoi ressemble une solution pour chaque entrée e
- La quantité à maximiser ou minimiser sur la solution

Exemple (3):

$$\text{PLUS COURT CHEMIN} : \begin{cases} \underline{\text{Entrée}} : \text{Un graphe G(V,E) non pondéré et } (s,t) \in V^2 \\ \underline{\text{Solution}} : \text{Un chemin simple C de } s \text{ vert } t \text{ dans G} \\ \underline{\text{Optimisation}} : \text{Minimiser le nombre d'arcs de C} \end{cases}$$

$$\text{PGCD} : \begin{cases} \underline{\text{Entrée}} : \ (p,q) \in \mathbb{N} \\ \underline{\text{Solution}} : \ d \in \mathbb{N} \text{ tel que } d|p \text{ et } d|q \\ \underline{\text{Optimisation}} : \text{Maximiser } d \end{cases}$$

Méthode

Tout problème d'optimisation peut être transformé en problème de décision par l'introduction d'un seuil.
Le principe pour un problème de maximisation est de transofrmer la recherche d'une solution de coût maximal et la recherche d'une solution plus grand que le seuil.

Exemple (4):
PGCD-DECI :
<u>Entrée :</u> $(p,q) \in \mathbb{N}, k \in \mathbb{N}$
<u>Question :</u> Y a-t-il un $d \in \mathbb{N}$ tel que $d|p$, $d|q$ et $d \geq k$?

PCC-DECI :
<u>Entrée :</u> G = (V,E), $(s,t) \in V^2, k \in \mathbb{N}$
<u>Question :</u> Y a-t-il un chemin de s vers t dans G de longueur en terme de nombre d'arêtes $\leq k$?

Remarque

Techniquement, pour que cette transformation crée bien un problème de décision il faut que la fonction de coût arrive dans un ensemble dont les éléments sont représentables de manière fini (par exemple $\mathbb{N}$ ou $\mathbb{Q}$ sont acceptés, mais pas $\mathbb{R}$).

II Modèle de calcul

But : Répondre à la question : Qu'appelle-t-on un « algorithme » ?

II.1 Modèle de calcul choisi par le programme

On se contente du modèle suivant : Un algorithme est un code (= une chaîne de caractères) en C ou en Ocaml qui s'exécute sur une machine avec une mémoire infinie

🔥 Parenthèse culture G

Il existe différents modèles de calcul (façon de définir ce qu'est un algorithme) : machines de Turing, λ-calcul, fonctions récursives, automates cellulaires…

Thèse de Church-Turing :
Tous ces modèles engendrent le même ensemble de fonctions calculables.
Tout modèle de calcul qui permet de calculer les mêmes fonctions que celles calculables avec les machines de Turing est dit « Turing-complet ». La plupart des langage de programmation sont Turing-complet donc la définition du programme est bien un modèle de calcul valable.

Exemple de langage non Turing-complet : Gallina.

Points commun des modèles Turing-complet :

- Un nombre fini de commandes élémentaires
- La possibilité d'enchaîner des commandes élémentaires.
- De la « mémoire » infinie.

Un autre point commun aux modèles Turing-complet est l'existence d'une **machine universelle**

ℹ Définition

Une **machine universelle** pour un modèle de calcul donné est un objet $M_u \in O = \{\text{objets décrits par le modèle de calcul}\}$ qui permet de simuler le comportement de n'importe quel élément de O

Exemple (5):

- PyPy est un interpréteur Python écrit en Python, c'est donc une machine universelle pour le modèle de calcul {codes Python qui s'exécutent sur machine à mémoire finie}
- CPython est un interpréteur Python écrit en C : ce n'est pas une machine universelle

⚠ Avertissement

`let universel f x = f x` n'est pas une machine universelle en Ocaml car elle ne prend pas le code source de la fonction f mais son nom.
Une machine universelle en Ocaml devrait avoir comme type `string` → `string -> string` avec le code source, l'encodage de l'argument et l'encodage du résultat

III Décidabiltié

III.1 Définition

Définition

Un problème de décision est **décidable** si la fonction $E \to \mathbb{B}$ est calculable. C'est à dire qu'il existe un algorithme *(= un élément défini par le modèle de calcul choisi)* qui termine toujours et qui répond correctement.

Exemple (6):

PGCD-DECI est décidable par l'algorithme d'Euclide.
PREMIER est décidable.

Remarque

Dans le contexte de la décidabilité, la complexité des algortihmes n'as aucun intérêt.

Proposition 1: Il existe une infinité non dénombrable de problèmes indécidables.

Démonstation:
Le nombre de problèmes est plus grand que le cardinal de $\{f : \mathbb{N} \to \mathbb{B}\}$, qui est indénombrable car il est en bijection avec $\mathscr{P}(\mathbb{N})$. La bijection en question est $\varphi : (f : \mathbb{N} \to \mathbb{B}) \mapsto \{x \in \mathbb{N} \mid f(x) = \top\}$.

Il y a moins de problèmes décidables que d'algorithmes.
En effet, un algorithme est un code source en Ocaml donc
$|\{\text{algorithme}\}| = |\{\text{codes sources}\}| = |\bigcup_{n \in \mathbb{N}} \{\text{codes source de taille } n\}|$.
Ainsi, c'est une union dénombrabre d'ensembles finis donc l'ensemble d'algorithmes est dénombrable.

III.2 Problème de l'arrêt

<u>**VERSION 1**</u> *(rigoureuse mais chiante)*

Définition

Le **problème de l'arrêt** est :
<u>Entrée :</u> le code d'une fonction f, l'encodage d'une entrée x pour f
<u>Question :</u> L'exécution de f sur x se termine-t-elle ?

Théorème 1: ARRÊT est indécidable.

Démonstation: Par l'absurde, ARRÊT est décidable.
Alors il existe une fonction arrêt en Ocaml de type `string → string → bool` qui décide ARRÊT.
À la place de donner à arrêt directement le code source de f, on lui donne le nom du fichier dans lequel on écrit le code source de f (quitte à rajouter une étape de lecture dans un fichier pour passer de (nom du fichier) à (code source)).

On écrit dans le fichier `paradoxe.ml` le code suivant :

```
let arret code arg = <code de arret>
if arret "paradoxe.ml" "" then
    while true do () done;
```

Exécutons ce code source sur l'entrée `""`.

- Si l'exécution termine alors, alors `arret "paradoxe.ml" ""` termine et renvoie vraie, donc on entre dans la boucle infini, et l'exécution ne termine pas.
- Si elle ne termine pas, alors `arret "paradoxe.ml" ""` termine et renvoie faux, donc on n'entre pas dans la boucle infini, et l'exécution termine.

L'exécution termine si et seulement si elle ne termine pas, ce qui est contradictoire, donc par l'absurde arrêt n'existe pas et ARRÊT est indécidable.

VERSION 2 *(avec abus mais partique et attendue en général)*

Définition

Le **problème de l'arrêt** est :
Entrée : une fonction f, un argumeent x pour f
Question : L'exécution de f sur x se termine-t-elle ?

Théorème 2: ARRÊT est indécidable.

Démonstation: Par l'absurde, ARRÊT est décidable.
Alors il existe une fonction arret en Ocaml de type `('a → 'b) → 'a → bool` qui décide ARRÊT. Ainsi, arrêt f x termine toujours et arrêt f x = vraie $\iff$ f x termine.

Si arret existe, on peut alors construire la fonction paradoxe : `unit → unit` suivante :

```
let paradoxe () =
      if arret paradoxe () then
```

```
            while true do () done

OU

let rec paradoxe () =
    if arret paradoxe () then paradoxe ()
```

- Si paradoxe () termine, alors arret paradoxe () s'evalue à true, on entre dans la boucle infinie et donc paradoxe () ne termine pas.
- Si paradoxe () ne termine pas, alors arret paradoxe () termine et s'évalue à faux, on n'entre pas dans la boucle infinie et donc paradoxe () termine.

Ainsi, paradoxe () termine $\iff$ paradoxe () ne termine pas, ce qui est contradictoire, donc arret n'existe pas et ARRÊT est indécidable.

III.3 Réductions

Idée

Pour montrer que P est indécidable, on peut essayer de montrer que si on sait résoudre P alors on sait résoudre ARRÊT.

Définition

On dit qu'un problème A **se réduit à** un problème B si et seulement si il existe une fonction $g : E_A \to E_B$ (avec E_A l'ensemble des instances (entrées) de A,...) telle que pour tout $e \in E_A$, $f_A(e) = f_B(g(e))$
où $f_A : E_A \to \mathbb{B}$ est la fonction sous-jacente au problème A (de même pour f_B)

On note $A \leq B$

Remarque

Si $A \leq B$ cela signifie que si on sait résoudre B alors on sait résoudre A.

Proposition 2: Si $A \leq B$:
- Si B est décidable A l'est aussi
- Si A est indécidable, B l'est aussi

Démonstation: Supposons $A \leq B$

- Supposons B décidable, un algo qui permet de résoudre A est :

- calculer g(e) avec g la fonction fournie par $A \leq B$
- calculer $f_B(g(e))$ (possible car f_B est calculable car B est décidable).

Cet algorithme termine toujours et renvoie vrai ssi e est positif pour A.

- Contraposé de précédemment

Exemple (7):

$$\text{ARRET_VIDE} : \begin{cases} \underline{\text{Entrée :}} \text{ Une fonction } f \\ \underline{\text{Question :}} \text{ Est-ce que l'exécution de } f \text{ sans argument termine ?} \end{cases}$$

ARRET_VIDE est indécidable :
Supposons par l'absurde que ARRET_VIDE est décidable.
Cela signifie qu'il existe une fonction OCaml av : `('a → 'b) → bool`.
Alors on peut écrire la fonction suivante :

```
let arret f x =
  let g () = f x in
  av g
```

Cette fonction décide le problème de l'arrêt car
- arret f x termine toujours
- arret f x renvoie true si f x termine
- arret f x renvoie faux si f x ne termine pas

On a monté que ARRET $\leq$ ARRET_VIDE, donc par indécidabilité de ARRET, on déduit celle de ARRET_VIDE.

Exemple (8):

$$\text{EQUIV} : \begin{cases} \underline{\text{Entrée :}} \text{ Deux fonction } f \text{ et } g \\ \underline{\text{Question :}} \text{ Est-ce que } f \text{ et } g \text{ sont équivalentes ?} \end{cases}$$

C'est à dire :
- terminent sur les mêmes entrées.
- ne terminent pas sur les mêmes entrées.
- pour les entrées où elles terminent, renvoient la même chose.

est indécidable.

Supposons par l'absurde que EQUIV est décidable.
Par définition, il existe une fonction equiv : `('a → 'b) → ('a → 'b) → bool` qui termine sur toute entrée et telle que equiv f g vaut true si et seulement si (f, g) est une instance positive de EQUIV.

Construisons alors :

```
let array f x =
  let f1 () = f x in
  let rec f2 () = f2 () in
  not (equiv f1 f2)
```

- arret termine sur toute entrée car c'est le cas de equiv
- Si f x termine, f1 termine sur toute entrée. De puis f2 ne termine sur aucune entrée, donc equiv f1 f2 renvoie false et arret f x renvoie true.
- Si f x ne termine pas, f1 ne termine sur aucune entrée. De plus f2 ne termine sur aucune entrée, donc equiv f1 f2 renvoie true et arret f x renvoie false.

Ainsi, arret décide le problème de l'arrêt, qui est indécidable, ce qui est contradictoire. EQUIV est donc indécidable.